PRACTICAL-8

Aim: To Implementation of Graph and Searching (DFS and BFS)

Algorithm:

Practical-8

Algorithm: (BFS)

Input: no of vertices, edges, Startingvertices
Output: Bfs Traversal
Step 1: Start
Step 2: Input Graph G and starting node S
Step 3: Create an empty queue Q & an empty visited list V
Step 4: While Q is not empty, repeat:
Step 5: Dequeue
Q: for each visited neighbor of N:
- mark it as visited
- insert it into Q
Step 6: If Q is empty, Stop loop
Step 7: Output in visited sequence
Step 8: Stop

Algorithm: (DFS)

Input: no. of vertices, edges, starting vertices
Output: DFS Traversal
Step 1: Start
Step 2: Input graph G & starting node S
Step 3: Creat empty stack ST & empty visited list V
Step 4: Push S into ST
Step 5: While ST is not empty, repeat:
- POP the to node
- if N is not visited
  - mark N visited
  - add N to output
- Push all unvisited neighbours

Step 6: if ST is empty, Stop the loop
Step 7: Output the visited sequence
Step 8: Stop

Code:

```
#include <iostream>
using namespace std;

class Graph
{
public:
   int adj[10][10];   // Adjacency matrix
   int visited[10];   // Visited array
   int n;             // Number of vertices

   // Get Input
   void getInput()
   {
      cout << "Enter number of vertices : ";
      cin >> n;

      cout << "Enter adjacency matrix (" << n << "x" << n << ") :" << endl;

      for (int i = 0; i < n; i++)
      {
         for (int j = 0; j < n; j++)
         {
            cin >> adj[i][j];
         }
      }
   }

   // Reset Visited Array
   void resetVisited()
   {
      for (int i = 0; i < n; i++)
      {
         visited[i] = 0;
      }
   }

   // BFS
   void BFS(int start)
   {
      int queue[10];
      int front = 0;
      int rear = 0;

      resetVisited();
```

```
    // Add starting vertex to queue
    queue[rear++] = start;
    visited[start] = 1;

    cout << "BFS Traversal : ";

    while (front < rear)
    {
        // Remove vertex from queue
        int node = queue[front++];

        cout << node << " ";

        // Visit all unvisited neighbours
        for (int i = 0; i < n; i++)
        {
            if (adj[node][i] == 1 && visited[i] == 0)
            {
                queue[rear++] = i;
                visited[i] = 1;
            }
        }
    }

    cout << endl;
}

// DFS
void DFS(int node)
{
    visited[node] = 1;

    cout << node << " ";

    for (int i = 0; i < n; i++)
    {
        if (adj[node][i] == 1 && visited[i] == 0)
        {
            DFS(i);
        }
    }
}

// Start DFS
void startDFS(int start)
{
```

```
        resetVisited();

        cout << "DFS Traversal : ";

        DFS(start);

        cout << endl;
    }

    // Display Adjacency Matrix
    void displayMatrix()
    {
        cout << "\nAdjacency Matrix:" << endl;

        for (int i = 0; i < n; i++)
        {
            for (int j = 0; j < n; j++)
            {
                cout << adj[i][j] << " ";
            }

            cout << endl;
        }
    }
};

int main()
{
    Graph g;

    g.getInput();

    g.displayMatrix();

    int start;

    cout << "\nEnter starting vertex : ";
    cin >> start;

    cout << endl;

    g.BFS(start);

    g.startDFS(start);

    return 0;
```

}

Output:

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Desktop\dda-practicals> g++ practical-8.cpp -o practical-8.exe
PS C:\Desktop\dda-practicals>  C:\Desktop\dda-practicals/practical-8.exe
 Enter number of vertices : 5
 Enter adjacency matrix (5x5) :
 0 1 1 0 0
 1 0 0 1 0
 1 0 0 0 1
 0 1 0 0 1
 0 0 1 1 0

 Adjacency Matrix:
   0 1 2 3 4
 0 0 1 1 0 0
 1 1 0 0 1 0
 2 1 0 0 0 1
 3 0 1 0 0 1
 4 0 0 1 1 0

 Enter starting vertex : 0

 BFS Traversal : 0 1 2 3 4
 DFS Traversal : 0 1 3 4 2
PS C:\Desktop\dda-practicals>
```

Time complexity:

Time complexity: (BFS)

while (front < rear)

for (int i=0; i<n; i++)

even if the starting vertex has no neighbours, the Program checks its entire row

V → one vertex

V x V = V^2 → up to V vertices

Best case = $O(V^2)$

The matrix still has to be scanned for the vertices

V x V = V^2

average case = $O(V^2)$

If the graph is highly connected, BFS processes all V vertices & scans V

V x V = V^2

(DFS) Worst case = $O(V^2)$

for (int i=0; i<n; i++) → for every visited vertex

Best case :

V x V = V^2

$O(V^2)$

average case :

V x V = V^2

$O(V^2)$

Worst case :

V x V = V^2

$O(V^2)$

Result:

The Graph Traversal problem was successfully implemented using Breadth First Search (BFS) and Depth First Search (DFS). The adjacency matrix was used to represent the graph, and the traversal was performed from the given starting vertex. The BFS and DFS traversals were successfully obtained.